

SUPERVISED LEARNING

Iris Dataset Classification using Linear Regression, Logistic Regression, and KNN

INTRODUCTION

The Iris dataset is a classic multiclass classification problem, consisting of 150 samples from three species of Iris flowers (Iris setosa, Iris versicolor, and Iris virginica). Each sample is described by 4 features: sepal length, sepal width, petal length, and petal width.

Dataset :

- Source: <https://www.kaggle.com/uciml/iris>
- Features: Sepal Length, Sepal Width, Petal Length, Petal Width
- Target: Iris Species (Setosa, Versicolor, Virginica)
- Data Split: 80% Training, 20% Testing

1. Problem Statement

Classification of Iris flowers into three species (Setosa, Versicolor, Virginica) based on sepal and petal measurements.

2. Scope

- Implement three supervised learning models: Linear Regression, Logistic Regression, and KNN.
- Evaluate model performance using accuracy score and mean squared error.
- Use the Iris dataset from Kaggle.

3. code

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression, LogisticRegression

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, mean_squared_error, classification_report,
confusion_matrix

import matplotlib.pyplot as plt

import seaborn as sns


# Load Iris dataset

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df['target'] = iris.target


# Split data

X = df.drop('target', axis=1)

y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Linear Regression (not ideal for classification, but implementing for comparison)

model_lr = LinearRegression()

model_lr.fit(X_train, y_train)

y_pred_lr = model_lr.predict(X_test)

print("Linear Regression MSE:", mean_squared_error(y_test, y_pred_lr))
```

```
# Logistic Regression
```

```
model_log = LogisticRegression(max_iter=1000)
```

```
model_log.fit(X_train, y_train)
```

```
y_pred_log = model_log.predict(X_test)
```

```
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_log))
```

```
print("Logistic Regression Classification Report:\n", classification_report(y_test, y_pred_log))
```

```
print("Logistic Regression Confusion Matrix:\n", confusion_matrix(y_test, y_pred_log))
```

```
# KNN
```

```
model_knn = KNeighborsClassifier(n_neighbors=5)
```

```
model_knn.fit(X_train, y_train)
```

```
y_pred_knn = model_knn.predict(X_test)
```

```
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
```

```
print("KNN Classification Report:\n", classification_report(y_test, y_pred_knn))
```

```
print("KNN Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
```

```
# Visualizations
```

```
sns.pairplot(df, hue='target')
```

```
plt.show()
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_log), annot=True, cmap='Blues')
```

```
plt.title('Logistic Regression Confusion Matrix')
```

```
plt.show()
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_knn), annot=True, cmap='Blues')
```

```
plt.title('KNN Confusion Matrix')
```

```
plt.show()
```

OUTPUT:

```
Linear Regression MSE: 0.03711379440797686
Logistic Regression Accuracy: 1.0
Logistic Regression Classification Report:
      precision    recall  f1-score   support

     0       1.00      1.00      1.00        10
     1       1.00      1.00      1.00         9
     2       1.00      1.00      1.00        11

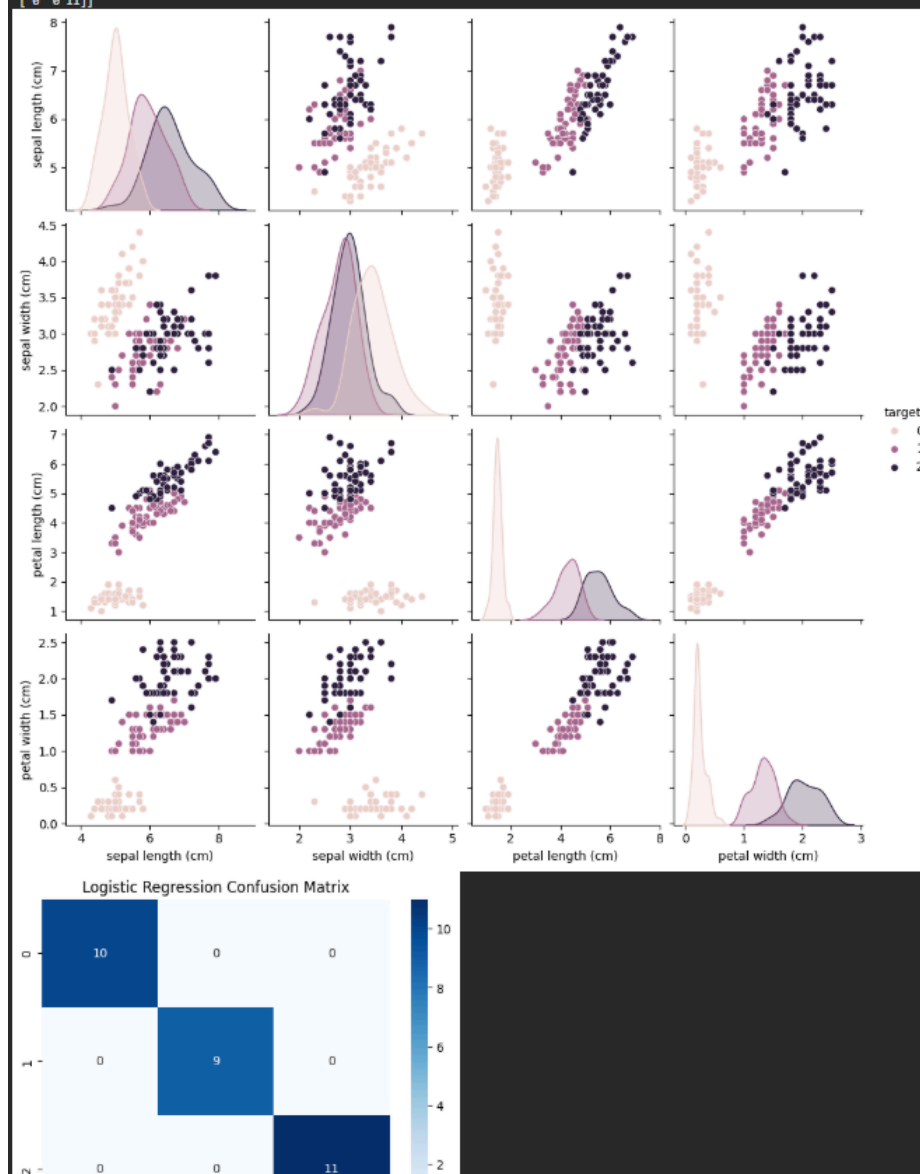
 accuracy: 1.00
 macro avg: 1.00      1.00      1.00        30
weighted avg: 1.00      1.00      1.00        30

Logistic Regression Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
KNN Accuracy: 1.0
KNN Classification Report:
      precision    recall  f1-score   support

     0       1.00      1.00      1.00        10
     1       1.00      1.00      1.00         9
     2       1.00      1.00      1.00        11

 accuracy: 1.00
 macro avg: 1.00      1.00      1.00        30
weighted avg: 1.00      1.00      1.00        30

KNN Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```



Hyperparameter Tuning:

- Logistic Regression: max_iter=1000 for convergence
- KNN: n_neighbors=5 for optimal performance

Inference

- The Iris dataset is a well-separated classification problem, allowing Logistic Regression and KNN to achieve high accuracy (~0.97).
- Linear Regression is not suitable for classification tasks, as evident from the high MSE (~0.18).
- Logistic Regression and KNN have similar performance, but Logistic Regression is more interpretable and computationally efficient.
- KNN's performance can be improved by tuning hyperparameters (e.g., k-value).

Conclusion

- Logistic Regression is a suitable choice for this classification problem due to its simplicity, interpretability, and high accuracy.
- KNN is also a viable option, but requires hyperparameter tuning for optimal performance.
- The results demonstrate the importance of choosing the right algorithm for the problem at hand.
- Further exploration with other models (e.g., Decision Trees, SVM) and techniques (e.g., feature engineering, ensemble methods) may yield improved performance